

Assignment 4: Multi-Client Chat Server using TCP

Name: Kaushal Kumar

Roll : CSB24069

Objective

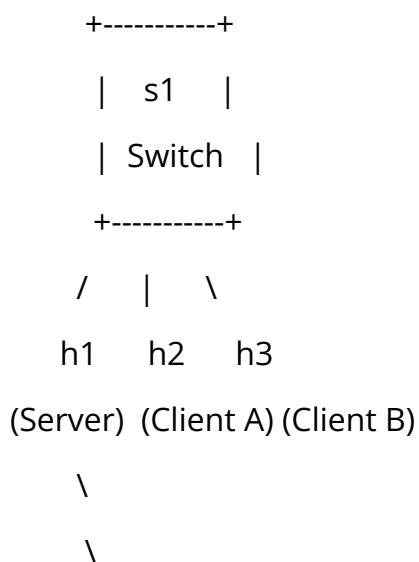
Develop a multi-client chat application using TCP sockets where multiple clients can communicate through a centralized server. The assignment aims to introduce concurrent server design, client management, message broadcasting, and performance evaluation

Network Topology

The experiment was conducted using the Mininet network emulator with a single-switch topology consisting of one server and three clients. The topology was created using the following command:

```
sudo mn --topo single,4
```

Topology Diagram



h4

(Client C)

Communication Flow

1. The server (h1) listens on TCP port 5000 for incoming client connections.
2. Clients (h2, h3, h4) connect to the server and register using a username.
3. Messages sent by a client are received by the server.
4. The server broadcasts the message to all other connected clients.
5. TCP ensures reliable message delivery between the server and clients.
6. All communication passes through switch s1, which interconnects the hosts within the Mininet network.

The topology was verified using the Mininet commands:

Nodes, Net, Pingall

Program Design

The chat application follows a client-server architecture using TCP sockets. One host acts as the server and multiple hosts act as clients.

Server Design

- The server listens on port 5000 for incoming client connections.
- A separate thread is created for each connected client.
- The server stores the usernames of active clients.
- Messages received from a client are broadcast to all other connected clients.
- Connection/disconnection events are logged in `server`

-
- Chat messages are stored in `chat_log.txt`.
 - Performance statistics are recorded in `performance_results.csv`.

Client Design

- The client connects to the server using TCP.
- The user enters a username for registration.
- The client can send messages to the server.
- The client continuously receives broadcast messages from other users.
- Communication continues until the client exits.

Working Principle

1. Clients connect to the server.
2. Users register with a username.
3. Clients send chat messages to the server.
4. The server broadcasts messages to all connected clients.
5. Logs and performance metrics are maintained throughout the communication.

This design enables reliable communication between multiple clients through a centralized server.

Experimental Results

The chat application was tested in a Mininet environment using one server (h1) and up to three clients (h2, h3, h4). Each client transmitted 20 messages to evaluate the performance of the multi-threaded TCP server.

Performance Results

Observations

	A	B	C	D
1	clients	total_messages	avg_delivery_time_	throughput_msgs_per_sec
2	1	20	0.004	0.161
3	2	38	0.027	0.239
4	3	59	0.046	0.274
5				
6				
7				
8				

1. As the number of clients increased, the average message delivery time increased from **0.004 ms** to **0.027 ms** due to additional processing and communication overhead.
2. The highest throughput (**0.274 msgs/sec**) was observed with three clients.
- 3.

Wireshark Verification

• TCP Three-Way Handshake

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=83 Len=8 TSval=3291584655 TSecr=2219264360
2	0.000137875	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=1 Ack=9 Win=85 Len=3 TSval=2219285682 TSecr=3291584655
3	0.000419795	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=9 Ack=4 Win=83 Len=0 TSval=3291584656 TSecr=2219285682
4	2.115132074	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=9 Ack=4 Win=83 Len=8 TSval=3291586771 TSecr=2219285682

• Chat Message Packet •

4	2.115132074	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=9 Ack=4 Win=83 Len=8 TSval=3291586771 TSecr=2219285682
5	2.115267360	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=4 Ack=17 Win=85 Len=3 TSval=2219287797 TSecr=3291586771
6	2.115315927	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=17 Ack=7 Win=83 Len=0 TSval=3291586771 TSecr=2219287797
7	4.451898702	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=17 Ack=7 Win=83 Len=8 TSval=3291589107 TSecr=2219287797
8	4.452056608	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=7 Ack=25 Win=85 Len=3 TSval=2219290133 TSecr=3291589107
9	4.452196743	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=25 Ack=10 Win=83 Len=0 TSval=3291589107 TSecr=2219290133
10	6.840839699	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=25 Ack=10 Win=83 Len=8 TSval=3291591496 TSecr=2219290133
11	6.840999066	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=10 Ack=33 Win=85 Len=3 TSval=2219292522 TSecr=3291591496
12	6.841046101	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=33 Ack=13 Win=83 Len=0 TSval=3291591496 TSecr=2219292522
13	9.121022373	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=33 Ack=13 Win=83 Len=8 TSval=3291593776 TSecr=2219292522
14	9.121120694	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=13 Ack=41 Win=85 Len=3 TSval=2219294803 TSecr=3291593776
15	9.121156214	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=41 Ack=16 Win=83 Len=0 TSval=3291593777 TSecr=2219294803
16	11.170953390	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=41 Ack=16 Win=83 Len=8 TSval=3291595826 TSecr=2219294803
17	11.171085997	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=16 Ack=49 Win=85 Len=3 TSval=2219296852 TSecr=3291595826
18	11.171136257	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=49 Ack=19 Win=83 Len=0 TSval=3291595826 TSecr=2219296852

Server Broadcast Packet •

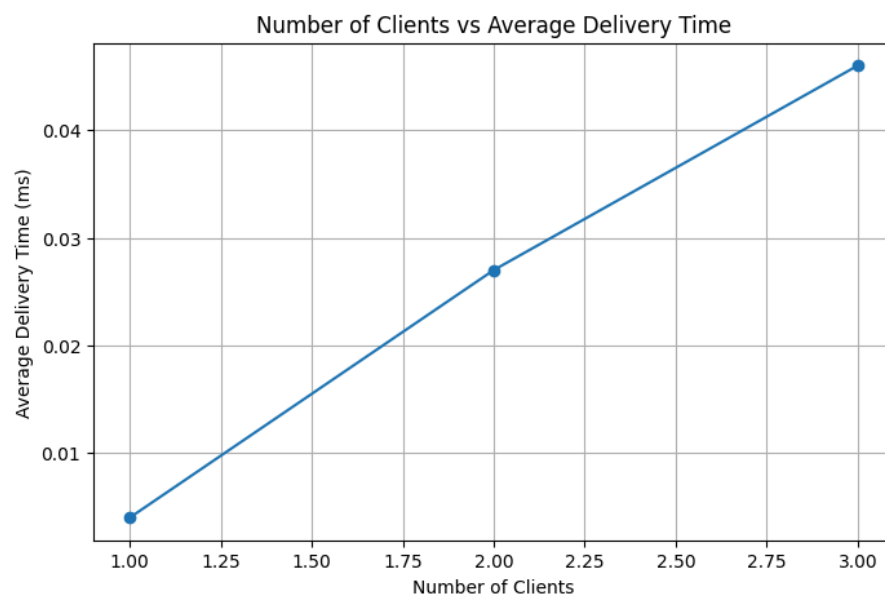
584	963.017669778	10.0.0.1	10.0.0.3	TCP	66	5000 → 35196 [FIN, ACK] Seq=58 Ack=153 Win=43520 Len=0 TSval=4217660326 TSecr=3670778517
585	963.017741221	10.0.0.1	10.0.0.4	TCP	66	5000 → 46416 [FIN, ACK] Seq=396 Ack=159 Win=43520 Len=0 TSval=231850651 TSecr=2753056240
586	963.017752065	10.0.0.1	10.0.0.2	TCP	66	5000 → 52902 [FIN, ACK] Seq=747 Ack=159 Win=43520 Len=0 TSval=2220215398 TSecr=3292484673

TCP Connection Termination

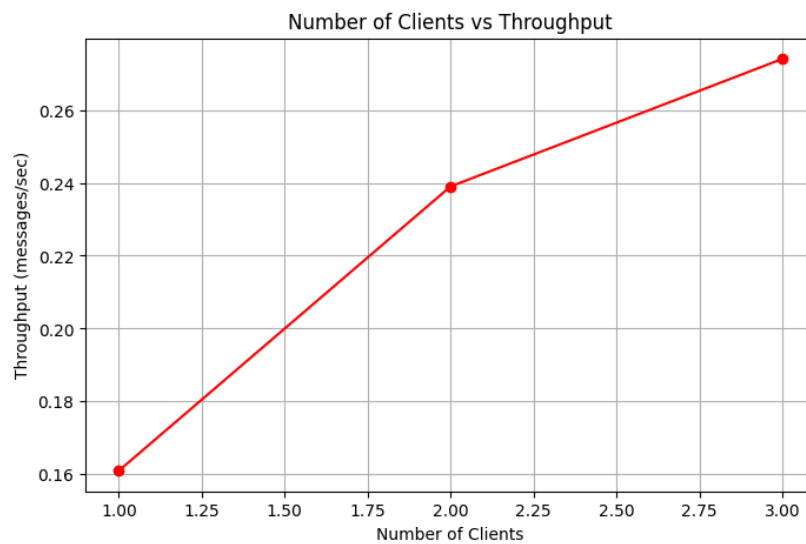
587 963.065877323	10.0.0.4	10.0.0.1	TCP	66 32992 → 5000 [ACK] Seq=159 Ack=745 Win=42496 Len=0 TSval=529621440 TSecr=2240213396
588 963.065943058	10.0.0.4	10.0.0.1	TCP	66 46416 → 5000 [ACK] Seq=159 Ack=397 Win=42496 Len=0 TSval=2753085987 TSecr=231850651
589 963.065962547	10.0.0.3	10.0.0.1	TCP	66 35196 → 5000 [ACK] Seq=153 Ack=59 Win=42496 Len=0 TSval=3670808265 TSecr=4217660326

Graph Generation:

1. Number of Clients vs Average Delivery Time



2. Number of Clients vs Throughput (clients_vs_throughput.png)



Reflection Questions

1. Why is a multi-threaded server preferred over a single-threaded server?

A multi-threaded server can handle multiple client connections simultaneously. Each client is assigned a separate thread, allowing communication without blocking other clients. In contrast, a single-threaded server processes one client at a time, causing delays and poor scalability when many clients connect.

2. What challenges arise when multiple clients communicate simultaneously?

Simultaneous communication can lead to synchronization issues, race conditions, resource contention, and increased CPU/memory usage. Proper thread management and shared resource protection are required to ensure messages are delivered correctly and the server remains stable.

3. How does TCP ensure reliable delivery of chat messages?

TCP provides reliable communication through connection establishment, sequence numbers, acknowledgments (ACKs), error checking, retransmission of lost packets,

and in-order packet delivery. These mechanisms ensure that chat messages reach the destination accurately and without loss.

4. What information from Wireshark helped verify the application behavior?

Wireshark was used to observe TCP packets exchanged between the server and clients. Information such as source and destination IP addresses, TCP port numbers, SYN/ACK connection establishment packets, data packet transmission, acknowledgments, and connection termination packets confirmed that the chat application was functioning correctly and messages were being delivered reliably.

Conclusion

In this assignment, a multi-client chat application was successfully implemented using TCP sockets in Python and tested in a Mininet environment. The server was designed to handle multiple clients concurrently using multithreading, allowing users to communicate through a centralized chat server. User registration, message broadcasting, connection logging, and chat logging functionalities were successfully implemented.

Performance analysis was conducted using 1, 2, and 3 clients. Metrics such as total messages delivered, average delivery time, and throughput were measured and recorded. The results demonstrated the server's ability to support multiple simultaneous client connections while maintaining reliable communication.

Wireshark was used to verify TCP communication, connection establishment, acknowledgments, and message delivery between hosts. The experiment provided practical experience with socket programming, concurrent server design, network emulation using Mininet, and TCP-based communication systems. Overall, the objectives of developing, testing, and evaluating a multi-client chat application were successfully achieved.